

Episodio 1: Requerimientos, cambios y configuración

Fuente para el podcast de repaso del parcial de **Aprobación Directa** de Ingeniería y Calidad de Software (opción múltiple, a libro abierto en papel). Contiene los capítulos 14, 15 y 20 del resumen de estudio `resumen-ad.md`, con su numeración original: las referencias a otros capítulos remiten a ese resumen.

CONTENIDO

14 Los requerimientos y su desarrollo

20 La gestión de configuración (CM)

15 La gestión de requerimientos y de los cambios

14. Los requerimientos y su desarrollo

Según el NIST, que cita la guía de requisitos de la cátedra, los requisitos **incompletos, imprecisos o contradictorios** causan cerca del **70 % de los defectos** de una aplicación. El problema no suele ser que después no se puedan corregir, sino que, por falta de tiempo, el equipo **se precipita o supone**, y el costo del producto se multiplica. De ahí salen dos ideas que sirven para justificar respuestas: los requisitos tienen que ser **entendidos por todas las partes antes de construir**, y **no se congelan**: el producto evoluciona y sus requisitos también, así que la evolución hay que gestionarla, no negarla.

Requisito: algo que el producto debe hacer o una característica que debe tener; una condición o capacidad que el sistema tiene que cumplir. Se escribe en forma **tecnológicamente neutra**: dice **qué**, no **cómo**.

La **ingeniería de requisitos** tiene dos mitades. El **desarrollo** produce los requisitos; la **gestión** administra los que ya existen: sus cambios y su consistencia con el resto del proyecto. En CMMI son dos áreas: **RD**, de **nivel 3**, y **REQM**, de **nivel 2**. Este capítulo trata la primera mitad; el siguiente, la segunda. La guía de INTECO dice *requisito* y la traducción del CMMI dice *requerimiento*: son lo mismo.

14.1 Tipos de requisitos

Por nivel de abstracción, hay requisitos **de negocio** —objetivos, visión y alcance—, **de usuario** —las tareas que el sistema ejecuta cuando el usuario opera con él— y **de sistema** —las funcionalidades que satisfacen a los anteriores, base del diseño y de las pruebas—. A eso se suman las **restricciones**, que limitan las opciones del diseñador.

La distinción que más se pregunta es la de **funcionales**, que dicen **qué debe hacer** el producto, y **no funcionales**, que dicen qué **cualidades** debe tener: rapidez, fiabilidad, seguridad, usabilidad. La guía lo dice con una imagen: los funcionales hacen que el producto realice el trabajo; los no funcionales le dan **carácter** al trabajo.

ATENCIÓN Un resumen de alumnos dice que los no funcionales "no son requeridos pero son deseables". La guía dice lo contrario: son **tan importantes como los funcionales**, y a veces deciden la aceptación. "Los no funcionales son opcionales" es **falso**.

Lo que el cliente dice no viene ordenado, y el analista tiene que clasificarlo. Si el cliente describe una forma específica de interactuar con el sistema, eso es **una solución sugerida, no un requisito**.

14.2 El desarrollo de requisitos según la guía

La guía organiza el desarrollo en cuatro actividades.

La **obtención** busca los requisitos resolviendo las diferencias entre los involucrados. Bien hecha, produce requisitos completos, consistentes, **identificados de forma única**, priorizados, **claros y no ambiguos**, y **testeables**, para poder verificarlos y validarlos después. El analista tiene que sacar a la luz también lo que el usuario **no sabe que necesita**, porque lo tiene tan internalizado que lo olvida: capturarlo ahora es mucho más barato que cuando aparece con el producto en uso. Las técnicas son las entrevistas, las reuniones, la observación, los cuestionarios —útiles cuando la gente es mucha o importa el anonimato—, el brainstorming, los casos de uso y los prototipos.

La **definición** los escribe. Las reglas son concretas: sin pronombres, con cuidado en los adjetivos y **sin "debería"**, que da a entender que el requisito es opcional; en lenguaje de negocio, y con un **glosario** que evoluciona con el proyecto.

La **verificación** controla cada requisito antes de que entre a la especificación, en una **puerta de calidad** que custodian el responsable del análisis y el **técnico de pruebas**. Ahí el usuario agrega los **criterios de aceptación**, y se evitan las **fugas de requisitos**: los que aparecen en la especificación sin que nadie sepa de dónde vienen.

La **revisión de la especificación** controla el **conjunto**: que no falte ningún tipo de requisito y que no haya **conflictos** entre ellos —dos requisitos están en conflicto si la solución de uno impide implementar el otro—. La **priorización** la hace el cliente por **valor**, con la información de **costo, dificultad y riesgo** que le da el desarrollador; si el cliente no logra priorizar, decide el jefe de proyecto.

Las responsabilidades quedan así: el **jefe de proyecto** es responsable de todo el proceso; el **cliente aprueba** los requisitos; el **CCB aprueba los cambios** (capítulo 15); y **SQA revisa**, pero no aprueba.

Un **prototipo** es una simulación de los requisitos, y rinde cuando los usuarios no saben bien lo que necesitan. El **horizontal** muestra todas las pantallas casi sin lógica, para aclarar el alcance; el **vertical** desarrolla pocas funciones en profundidad. En ningún caso **reemplaza la especificación escrita**: la complementa.

14.3 RD – Desarrollo de requerimientos

Propósito de RD: producir y analizar los requerimientos **de cliente, de producto y de componente del producto**.

Los tres tipos existen porque el cliente habla en sus términos: éstos son los **requerimientos de cliente**. Para diseñar hay que traducirlos a términos técnicos, y así nacen los **de producto y de componentes**.

- **SG 1 Desarrollar los requerimientos de cliente.** SP 1.1 **Obtener las necesidades** · SP 1.2 **Desarrollar los requerimientos de cliente** (consolidar lo relevado, resolver conflictos entre interesados).
- **SG 2 Desarrollar los requerimientos de producto.** SP 2.1 **Establecer los requerimientos de producto y de componentes** · SP 2.2 **Asignar los requerimientos de componentes** · SP 2.3 **Identificar los requerimientos de interfaz**.
- **SG 3 Analizar y validar los requerimientos.** SP 3.1 **Establecer los conceptos operativos y los escenarios** · SP 3.2 **Establecer una definición de la funcionalidad requerida** · SP 3.3 **Analizar los requerimientos** (que

sean completos, factibles y **verificables**: acá cae el requisito vago o no medible) · SP 3.4 Analizar para alcanzar el **equilibrio** con costo, plazo y riesgo · SP 3.5 **Validar** los requerimientos.

ATENCIÓN En RD, toda opción en la que **el equipo decide solo** es incorrecta: elegir los dispositivos, completar con métricas estándar de la industria, "lo resolvemos en las pruebas". RD exige **confirmar con el cliente**. En el AD de 2024, ante un cliente que pedía acceso desde dispositivos móviles sin decir cuáles, la correcta fue definir una lista de dispositivos y sistemas operativos y **confirmarla con el cliente**. Ante requisitos de usabilidad vagos, la correcta fue reunirse con el cliente para definir **métricas** y actualizar la documentación.

RD tiene dos fronteras que se preguntan. **Con REQM**: RD documenta las relaciones entre requerimientos al refinarlos, pero **mantener la trazabilidad** es REQM SP 1.4. **Con VAL**: RD SP 3.5 valida **los requerimientos**, temprano y sobre prototipos o simulaciones, mientras que VAL valida **el producto**; probar el sistema con el cliente es VAL. Y como RD define desde el relevamiento las restricciones para verificar y validar, "la validación recién empieza al final del proyecto" es falso.

15. La gestión de requerimientos y de los cambios

Una vez definidos los requisitos —e incluso con el sistema ya implantado— aparecen necesidades nuevas, correcciones y mejoras. El cambio no es una falla del relevamiento: es inherente al software. La falla es no gestionarlo. El cliente le pide un ajuste por mail al programador, el programador lo hace esa tarde, y tres meses después nadie sabe por qué el sistema calcula distinto de lo que dice la especificación, ni qué otras partes tocó ese cambio.

La **gestión de requisitos** es el conjunto de actividades para **identificar, controlar y seguir** los requisitos y sus cambios, y asegurar la **consistencia** entre los requisitos y el sistema construido. Su actividad más importante es gestionar los **cambios**, y trabaja sobre una línea base.

Línea base: conjunto de productos de trabajo **revisado y acordado formalmente**, que sirve de base para el desarrollo posterior y que **sólo puede cambiarse mediante un procedimiento formal de control de cambios**.

Cambiar la línea base de requisitos es, por definición, un **cambio de alcance**.

15.1 El orden ante un cambio

El proceso de la guía para cada petición de cambio tiene un orden que se pregunta una y otra vez. **Primero se registra y se evalúa el impacto**: con la matriz de trazabilidad se ve qué otros requisitos, módulos y casos de prueba se afectan, y cuánto cuesta el cambio. **Después se valora**: si el impacto es asumible, se acepta; si no, se **negocia con el cliente**; si se rechaza, queda registrado. **Recién entonces** se modifican todos los productos afectados —la especificación, la matriz y, si hace falta, el plan—, se establece una **nueva línea base** y se obtiene la aprobación del cliente.

ATENCIÓN Toda opción que **implemente antes de evaluar el impacto** es incorrecta. En el AD de 2024, ante un cliente que en plena implementación pedía restricciones horarias que afectaban a varios subsistemas, la correcta fue analizar cómo afectan a los otros subsistemas y actualizar la matriz de trazabilidad **antes de proceder**. La trampa era "registrar el cambio, actualizar la matriz y **comenzar la implementación**": parece REQM, pero se saltea la evaluación.

En el mismo parcial, el cliente de un comercio electrónico quería registrar la navegación de los usuarios, lo que implicaba cumplir normas de protección de datos. El primer paso correcto fue **consultar con legales y privacidad** y reflejar el impacto en la matriz: las leyes son fuente de requisitos y de cambios como cualquier otra.

15.2 La trazabilidad

Trazabilidad bidireccional: asociación **en ambos sentidos** entre el requisito y su **origen** (hacia atrás) y entre el requisito y el diseño, el código y las pruebas (hacia adelante).

Hacia atrás responde **de dónde viene** el requisito y si tiene una fuente válida; hacia adelante, **qué se rompe si lo cambio** y si está cubierto por pruebas. Sirve, sobre todo, para **evaluar el impacto de un cambio**. Y tiene un costo: si actualizarla es muy trabajoso, los desarrolladores cambian el código sin actualizar los requisitos, que se vuelven inútiles para probar.

ATENCIÓN ¿Qué área **establece y mantiene** la trazabilidad entre los requerimientos y los otros productos de trabajo? **REQM**, SP 1.4.

15.3 REQM – Gestión de requerimientos

Propósito de REQM: gestionar los requerimientos de los productos y componentes del proyecto, e **identificar inconsistencias** entre esos requerimientos y los **planes y productos de trabajo** del proyecto.

Es un área de **nivel 2** con una sola meta, **SG 1 Gestionar los requerimientos**, y gestiona **todos** los requerimientos que el proyecto recibe o genera, vengan de donde vengan.

PRÁCTICA	CÓMO SE RECONOCE EN UN ENUNCIADO
SP 1.1 Obtener una comprensión de los requerimientos	Entender con quien lo pidió qué significa; canales oficiales, criterios de aceptación
SP 1.2 Obtener el compromiso sobre los requerimientos	Acordar con quienes implementan, o con el Sponsor, el impacto sobre un compromiso
SP 1.3 Gestionar los cambios de los requerimientos	Documentar la razón del cambio; " se está chequeando cómo afecta"
SP 1.4 Mantener la trazabilidad bidireccional	"Actualizar la matriz de trazabilidad "
SP 1.5 Identificar las inconsistencias entre el trabajo y los requerimientos	"El plan o el caso de uso no coincide con el requerimiento"

La diferencia entre SP 1.1 y SP 1.2 es **con quién**. La comprensión se logra con el **proveedor** del requerimiento: el cliente o el usuario que lo pide. El compromiso, con los **participantes del proyecto** que lo van a implementar. Acordar con el Sponsor posponer tres semanas la implementación para incorporar unos cambios es **REQM SP 1.2**: se negocia el impacto de un cambio sobre un compromiso existente.

15.4 Las solicitudes de cambio

La gestión de requisitos dice **qué** decidir ante un cambio; la **gestión de solicitudes de cambio** de RUP dice **cómo** se tramita, para que todo cambio pase por un procedimiento estándar, deje registro y se haga con efecto predecible.

Solicitud de cambio (CR): producto de trabajo enviado formalmente para rastrear las solicitudes de los interesados —funciones nuevas, mejoras, **defectos**, requisitos cambiados— con su estado e historial durante todo el ciclo de vida.

CCB (panel de control de cambios): grupo que supervisa el proceso de cambio, con representantes de **todas las partes interesadas**. En un proyecto chico puede ser **una sola persona**.

La reunión del CCB decide si cada CR es **válida** y si entra en el release actual. El camino feliz de una CR es **Enviado** → **Abierta** → **Asignada** → **Resuelta** → **Verificada** → **Cerrada**; los desvíos son **Pospuesta** (válida pero fuera del release actual), **Duplicada**, **Rechazada**, **Más información** y **La prueba ha fallado**. Sólo la reunión del CCB puede abrir una CR, y sólo el administrador de revisión del CCB puede cerrarla.

En CMMI, este proceso es CM (capítulo 20): **SP 2.1 Seguir las peticiones de cambio** y **SP 2.2 Controlar los elementos de configuración**. Las solicitudes de cambio no son de REQM, y no tratan sólo requisitos: también **defectos**.

15.5 REQM, RD o CM: cómo decidir

Las tres áreas giran alrededor de los requisitos, y separarlas es de lo más preguntado de la unidad. Dos pasos resuelven casi todo.

Primer paso: ¿el requisito ya está acordado? Si todavía se está relevando, aclarando, completando o validando, es **RD**. Si ya está en la línea base, seguí al segundo paso.

Segundo paso: ¿qué se hace con el requisito acordado?

SITUACIÓN DEL ENUNCIADO	ÁREA / PRÁCTICA
Llega un cambio y todavía se evalúa cómo afecta	REQM SP 1.3
Actualizar o consultar la matriz de trazabilidad	REQM SP 1.4
El plan o los casos de uso no coinciden con los requisitos vigentes	REQM SP 1.5
El cambio ya se decidió aceptar y se tramita: seguir la petición, ver qué modificar en los casos de uso	CM SP 2.1
Revisar y acordar formalmente un conjunto de artefactos como base del desarrollo	CM SP 1.3
Alguien externo al proyecto detecta cambios hechos sin seguir el procedimiento y escala	PPQA SP 2.1

El ejemplo canónico viene del cuestionario de la cátedra. "El cliente pidió una variante de descuento no acordada; en este momento se está chequeando con el contador cómo afecta al esquema impositivo" es **REQM SP 1.3**. "Ya se decidió aceptar la variante; ahora se chequean las modificaciones a realizar en los casos de uso" es **CM SP 2.1**.

ATENCIÓN El análisis de impacto aparece **en las dos áreas**. Lo que decide es **el momento**: si todavía se evalúa si conviene, REQM; si ya se aceptó y está en trámite, CM. Y decide el **contenido** del cambio, no quién lo pide: un cambio interno que **no altera ningún requisito** —el equipo refactoriza o corrige un defecto— se controla en **CM**; si altera un requisito, es REQM aunque lo proponga el equipo. La regla de alumnos "REQM trata los cambios que pide el cliente" sirve como atajo, porque casi siempre coincide.

20. La gestión de configuración (CM)

Un producto de software es un conjunto de piezas —requerimientos, diseño, código, casos de prueba, manuales y hasta las herramientas con que se construyó— que cambian todas, a ritmos distintos y en manos distintas. Cuando la software factory vende el mismo producto a varios clientes, cada uno con su versión, y el equipo corrige errores en paralelo, aparecen las frases con que la guía de la cátedra retrata a un proyecto sin gestión de configuración: "¿cuál es la versión que tiene el cliente?", "no puedo reproducir el problema en mi versión", "¿está corregido el error también en esa versión?". El riesgo principal es **entregar al cliente la versión incorrecta**, y detrás viene otro: **no poder recuperar una versión anterior** para mantener a un cliente que sigue usándola.

Gestión de configuración (CM): establecer y mantener la **integridad** de los productos de trabajo utilizando la **identificación** de configuración, el **control** de configuración, el **registro del estado** de configuración y las **auditorías** de configuración.

Integridad quiere decir saber **exactamente qué se le entregó a cada cliente** y conocer el estado y el contenido de cada versión. CM es un área **de soporte de nivel 2**: no produce nada propio, sino que pone bajo control lo que generan las demás.

20.1 Metas y prácticas

Las tres metas cuentan una historia en orden: se **crean** las líneas base, se las **mantiene** con cambios controlados, y se **documenta y audita** su integridad.

- **SG 1 Establecer líneas base.** SP 1.1 **Identificar** elementos de configuración · SP 1.2 Establecer un **sistema** de gestión de configuración · SP 1.3 Crear o liberar **líneas base**.
- **SG 2 Seguir y controlar los cambios.** SP 2.1 Seguir las **peticiones de cambio** · SP 2.2 **Controlar** los elementos de configuración.
- **SG 3 Establecer la integridad.** SP 3.1 Establecer **registros** de gestión de configuración · SP 3.2 Realizar **auditorías** de configuración.

20.2 Qué se pone bajo configuración

Elemento de configuración (EC): cualquier producto de trabajo —final o intermedio, entregable al cliente o interno— **cuyo cambio pueda resultar crítico** para el proyecto, y que se trata como **una entidad única**.

Ser EC no es una categoría aparte: un producto, un producto de trabajo interno o una herramienta pasan a ser EC cuando **se decide ponerlos bajo control**. Identificarlos (SP 1.1) es la base de todo lo demás: lo que no se identificó no se puede controlar, registrar ni auditar. Van bajo configuración los productos **usados por dos o más grupos**, los que **pueden cambiar**, los que **dependen entre sí** y los **críticos**: los planes, la especificación de requisitos y la matriz de trazabilidad, el diseño, el **código fuente**, los **casos y datos de prueba**, los manuales y todos los entregables.

ATENCIÓN Las herramientas también son elementos de configuración si hacen falta para reconstruir una versión: sin el lenguaje y el compilador **en la versión que se usó**, el código fuente no alcanza para regenerar el ejecutable.

La pregunta del año 2002. El AD de 2024 describió una software factory que desde 2002 desarrolla aplicaciones de escritorio en un lenguaje A; hace dos años sumó apps móviles en un lenguaje B; aplica un estándar de interfaces desde 2010; sus manuales eran PDF hechos con MS-Word 2002, y luego los reemplazó un módulo de manual on-line. La pregunta: "si estuviera en el año 2002, ¿qué debería quedar bajo la gestión de configuración para garantizar la evolución que se tuvo?". El criterio es **pararse en el año del enunciado** y poner todo lo que **existía entonces y hace falta para reconstruir** el software: la documentación del sistema, la app de escritorio, su **código fuente**, el **lenguaje A**, **MS-Word 2002** y los manuales en PDF. Lo que apareció después **no puede estar**: ni el lenguaje B —marcarlo dejó la pregunta en cero—, ni el estándar de interfaces, ni el módulo on-line. Queda sin confirmar si valían los manuales en .doc.

Desde la perspectiva de las **pruebas**, CM sirve para controlar la versión de los casos de prueba, **identificar qué versión del software se está probando** y seguir los cambios de los casos; no sirve para desarrollar casos nuevos.

20.3 Las líneas base

Línea base (LB): conjunto de productos de trabajo **revisado y acordado formalmente**, que sirve de base para el desarrollo posterior y que **sólo puede cambiarse mediante el procedimiento de control de cambios**.

Para entrar en una línea base, un producto tiene que estar **acabado** y **formalmente aprobado**, típicamente en una **revisión formal**. Crearla es la SP 1.3, y requiere la autorización del comité de cambios. La línea base más los cambios aprobados forman la **configuración vigente**.

Lo más importante que hace una línea base no está en la definición: **relaciona entre sí las versiones de los distintos artefactos**. Así lo muestra el ejemplo de la guía para un ciclo en cascada, donde las letras son versiones:

HITO (REVISIÓN FORMAL)	CONTENIDO DE LA LÍNEA BASE
Revisión de requisitos	ERS A
Revisión de diseño	ERS B · Diseño A
Disponibilidad de las pruebas	ERS C · Diseño B · Código A · Plan de pruebas A
Aceptación	ERS D · Diseño C · Código B · Plan de pruebas B · Manual de usuario A

Sin esa relación no hay forma de saber que el producto aceptado se armó con la versión D de la especificación y la B del código. Los tipos de línea base cambian según la fuente: la guía habla de **funcional**, **de desarrollo** y **de producto**; CMMI, de **funcional**, **asignada** y **del producto**.

20.4 Versiones, variantes y releases

Con varios clientes en versiones distintas hay que poder identificar la de cada uno, saber **en qué versión entró cada cambio** y **reconstruirla**.

Versión: variación **temporal** de un producto o de un EC: el mismo elemento, evolucionado en el tiempo. **Variante:** variación **espacial**: el mismo producto adaptado a otro ambiente, plataforma o lenguaje, que **coexiste** con los demás.

Una app que pasa de Android 1.1 a Android 4.3 sin cambiar su funcionalidad **no es una versión nueva, es una variante**. Dentro del número de versión, la **mayor** sube con un cambio de funcionalidad, y la **menor**, con correcciones o cambios chicos.

Release: versión de lanzamiento, en la que el software **se hace público**.

De ahí sale otra pregunta del AD de 2024: el **mínimo de releases** lo decide el ciclo de vida. En **cascada** es **uno**, porque hay una sola entrega al final. En un **incremental con n incrementos** son **n**, porque cada incremento se entrega. En el enunciado real el producto se llamaba "ALFA 3" y tenía 4 incrementos: la respuesta era **4**; el "3" es parte del nombre.

La **política de versionado** es la regla escrita que dice el **formato** de la versión, **con qué valor arranca** cada componente, **cuándo sube** cada uno y si los demás **se reinician**. Si el producto está hecho de módulos, además registra qué versión de cada módulo compone cada versión del producto: eso es la línea base. No hay que confundirla con la **convención de nombres** de los archivos, que es otra cosa; la de la cátedra es **MM-TTT-AAnn_nombre_vx_yy**, donde la versión mayor **x** vale 0 mientras el documento no es definitivo.

20.5 Cuánto sobrevive un esquema de versión

Un esquema de numeración tiene capacidad finita: con dos dígitos hay 99 valores, y cuando se agotan, el esquema "muere". El AD de 2024 pidió calcular cuántos años dura uno, y el método es este: contar los **valores** de cada componente; fijar su **frecuencia** anual; si el menor **se reinicia** cuando cambia el mayor, verificar que no se desborde dentro de un período del mayor; y si no se desborda, la supervivencia es **valores del mayor / frecuencia anual del mayor**.

El enunciado: versión **<x>.<y>**; **x** cambia con la funcionalidad, tiene dos dígitos y empieza en 01; **y** sube con cada corrección, tiene dos dígitos y **vuelve a 0 cuando cambia la funcionalidad**; la funcionalidad cambia **cada trimestre** y los errores se corrigen **cada quince días**.

¿Se desborda y? Un trimestre tiene unas 6 correcciones quincenales: y llega a 06 o 07, lejos de 99. Manda x.

Capacidad de x: de 01 a 99 → 99 versiones funcionales.

Tiempo: 4 cambios por año → 99 / 4 ≈ 24,75 años.

Respuesta: 24, la opción más cercana sin pasarse.

ATENCIÓN El distractor es dividir los 99 valores de **y** por las quincenas del año, como si no se reiniciara: da menos de cuatro años.

20.6 Los repositorios

No tiene sentido controlar igual el borrador que un programador tiene en su máquina y la versión entregada al cliente. Por eso el **sistema de gestión de configuración** (SP 1.2) —almacenamiento, procedimientos y herramientas— distingue tres repositorios:

REPOSITORIO	QUÉ CONTIENE	CONTROL
Dinámico (de desarrollo)	Lo que se está creando o revisando	Control de versiones , a cargo del desarrollador
Máster (controlado)	La línea base actual y sus cambios	Control de configuración completo
Estático	Las líneas base ya liberadas y archivadas	Control de configuración completo

En el día a día, un archivo no se modifica sin **desprotegerlo** (check-out) y vuelve como **versión nueva** al protegerlo (check-in). El sistema incluye también los permisos de acceso y las **copias de seguridad**, guardadas en un sitio distinto al de los originales.

ATENCIÓN **Control de versiones no es gestión de configuración formal.** Y poner bajo control los registros de **otra** área —los informes de PPQA, por ejemplo— es la práctica genérica **GP 2.6 Gestionar configuraciones** de esa área, no una práctica de CM.

20.7 El control de cambios y el comité

Los cambios son inevitables. Lo que CM garantiza es que **sólo entren los aprobados**, que se sepa **en qué versión y en qué línea base** entró cada uno, y que todos trabajen con las versiones correctas.

Comité de control de cambios (CCB): evalúa y aprueba o rechaza los cambios propuestos a los EC, asegura que se implementen los aprobados y autoriza las líneas base. En un proyecto chico puede ser el líder o una persona asignada.

Necesidad de cambio → solicitud sobre un EC → análisis de impacto → revisión del CCB
→ rechazada: se informa al solicitante
→ aprobada: se asigna, se planifica, se implementa, se prueba y se cierra

La solicitud la inicia **cualquiera, en cualquier momento**, y no trata sólo requerimientos: también **fallos y defectos**. El comité puede aceptar, modificar, rechazar o aplazar, y la decisión **siempre queda documentada**. Las dos prácticas de SG 2 se reparten el circuito: la **SP 2.1** registra la petición, analiza el impacto y la **sigue hasta su cierre**; la **SP 2.2** controla el elemento que cambia —autorización, check-in y check-out, registro del cambio y de su razón—. Las modificaciones en sí las hace el proceso técnico que corresponde; CM las registra y libera la nueva línea base.

20.8 El registro del estado y las auditorías

El **registro del estado** (SP 3.1) guarda el historial de cada EC con detalle suficiente para **recuperar versiones anteriores**, y dice **qué versión de cada EC constituye cada línea base** y qué diferencias hay entre líneas base sucesivas. Por eso "cada vez que se genera una compilación se genera un documento con las modificaciones respecto de la anterior" es SP 3.1.

Auditoría de configuración: verificación de que un EC, o el conjunto de EC que forman una línea base, se ajusta a un estándar o requerimiento especificado.

Se hace en **puntos clave** del ciclo de vida, y una auditoría exitosa es requisito para establecer la línea base del producto. CMMI distingue tres tipos: la **funcional** verifica que el EC, ya probado, logra los requerimientos de su línea base; la **física**, que el EC tal como fue construido es conforme con su documentación técnica; y la **de gestión de configuración**, que los registros son completos, consistentes y exactos.

ATENCIÓN La auditoría de configuración **no evalúa el producto en sí:** comprueba que esté **bien gestionado**, con todos los cambios registrados y trazabilidad entre cambios y productos. No es VER (¿cumple su especificación?) ni PPQA (¿se siguió el proceso de la organización?).

20.9 Cómo distinguir CM de las áreas vecinas

CM comparte vocabulario con REQM (los cambios), con PPQA (las auditorías y los estándares) y con OPD (los repositorios), y los ejercicios juegan con eso:

LO QUE DICE EL ENUNCIADO	ÁREA Y PRÁCTICA
Un conjunto de artefactos, acordados formalmente , pasa a ser la base del resto del desarrollo	CM SP 1.3
Llega un cambio y todavía se evalúa cómo afecta	REQM SP 1.3, no CM (capítulo 15)
El cambio ya se decidió aceptar y se siguen las modificaciones	CM SP 2.1
Se controla que los nombres de archivo cumplan las reglas del proyecto	CM SP 2.2
Personal externo controla que se cumpla un estándar de la organización	PPQA SP 1.2, no CM
Cada compilación genera un documento con las diferencias respecto de la anterior	CM SP 3.1
Se arma el repositorio organizacional de plantillas y procesos	OPD, no CM

El BP de versionado del AD de 2024. Una software factory vende productos a varios clientes y tiene una política que identifica la versión del producto final y la de cada artefacto, pero **cada artefacto tiene su número independiente y no se guarda la relación con las versiones de los otros**. La respuesta es que **sí** podría generar problemas de calidad, **porque si hay que volver a una versión anterior no hay información sobre la relación de versiones entre artefactos**: falta la línea base (SP 1.3) y el registro de qué versión de cada artefacto la compone (SP 3.1).

ATENCIÓN La opción "el problema principal es que no se almacenan en un **repositorio organizacional**" fue **incorrecta**: CM no exige un repositorio organizacional, eso es tema de OPD. También es falsa "es una implementación correcta de CM, dado que se conservan todas las versiones": las versiones sueltas no alcanzan.